A Programmer's Mental Model for Proving

Yazan Daba
yazandaba@outlook.com

***Abstract*—Averroes is a formally verified Lisp parser written in Lean. Its proofs cover lexer behavior, parser progress, and specific parsing cases. The approach follows one path through the control-flow graph, states the facts that define the case, and proves what the return value means. The four return paths of the number lexer illustrate the method, with the integer proof as a worked example. The paper also shows what it takes to reuse a path proof in a caller.**

***Index Terms*—Formal verification, Lean, lexing, parsing.**

I. START WITH A PATH

A control-flow graph shows the paths that execution can take through a function. The approach follows a path from the function's entry to an exit basic block representing the returned value. Each path is a separate proof problem with its own path condition, invariants, and guarantees about the result. The integer-path theorem `LexValidInteger` from Averroes [1] is the central example.

Two paths can both return `5` under different conditions. We reason about them separately first. If one statement captures what matters about both simple cases, we can combine them, much as we would when writing a function.

II. THE NUMBER LEXER

Listing 1 shows the complete `LexNumber` definition with line breaks and spelling adjusted for readability. On nonempty input it calls `span Char.isDigit` to split the characters into a leading digit run and what follows it. A dot after the run starts the decimal case. The next decision is whether there are digits after the dot.

The integer and valid decimal paths both return `TokenKind.Number`. They promise different token text. The empty and malformed decimal paths both return `TokenKind.Invalid`, with different messages and spans. Figures 1 to 4 separate these cases and point back to the listing.

The outer lexer calls `LexNumber` when the first character is a digit. The helper also handles empty input when called directly. Keeping that distinction visible makes it clear which inputs the caller must consider.

Listing 1. Number lexer adapted from Averroes [1]. Line numbers are local to this listing.

```
1  def LexNumber : (List Char) -> Nat -> Nat -> Token
2  | [] , line, column =>
3    Token.Create TokenKind.Invalid
4      (SourceLocation.Create
5        (SourcePosition.Create line column)
6        (SourcePosition.Create line column))
7      "[FATAL] INVALID EMPTY NUMBER LEXEME"
8  | fnum :: rest, line, column =>
9    let (number, remainder) := (fnum::rest).span Char.isDigit
10   if remainder[0]? == some '.' then -- floating point
11     let mantissa := (remainder.drop 1).takeWhile Char.isDigit
12     if mantissa.length == 0 then
13       Token.Create TokenKind.Invalid
14         (SourceLocation.Create
15           (SourcePosition.Create line column)
16           (SourcePosition.Create line
17             (column + (String.ofList number ++ ".").length)))
18         s!"malformed floating point literal '{String.ofList number}.'"
19     else
20       let float :=
21         (String.ofList number) ++ "." ++ (String.ofList mantissa)
22       Token.Create TokenKind.Number
23         (SourceLocation.Create
24           (SourcePosition.Create line column)
25           (SourcePosition.Create line (column + float.length)))
26         float
27   else
28     Token.Create TokenKind.Number
29       (SourceLocation.Create
30         (SourcePosition.Create line column)
31         (SourcePosition.Create line (column + number.length)))
32       (String.ofList number)
```

III. FOUR PATHS THROUGH LEXNUMBER

Each diagram shows the decisions and steps on one path from entry to return. The L labels point to lines in Listing 1. The scan operations are shown as single steps.

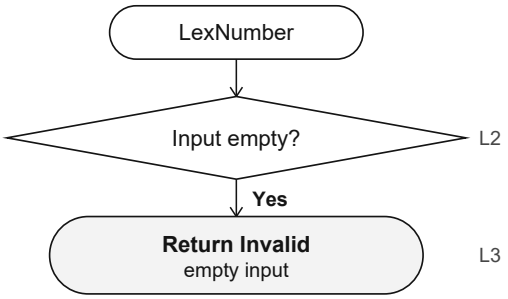


Fig. 1. Empty input. The helper returns an invalid token with a zero-width span.

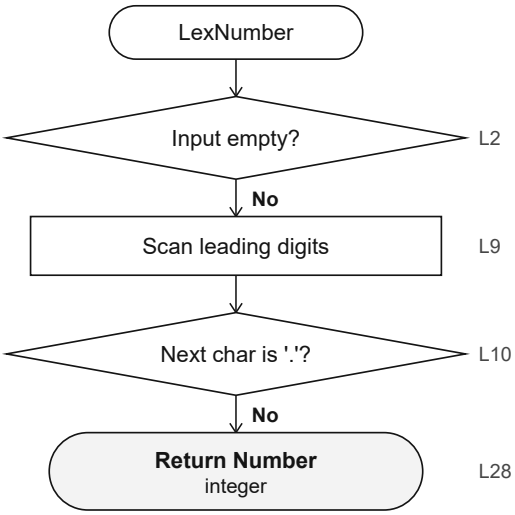


Fig. 2. Integer path. The token contains the leading digits. `LexValidInteger` describes this case.

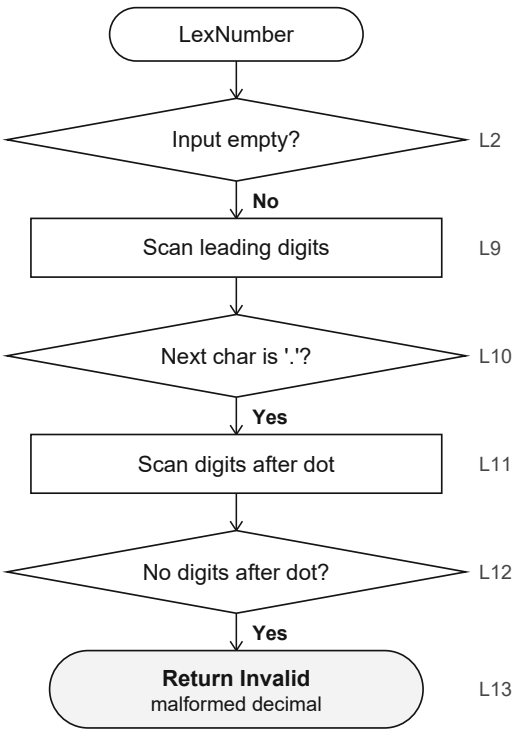


Fig. 3. Malformed decimal. With `123.x`, the span covers `123.` and the token text is a diagnostic.

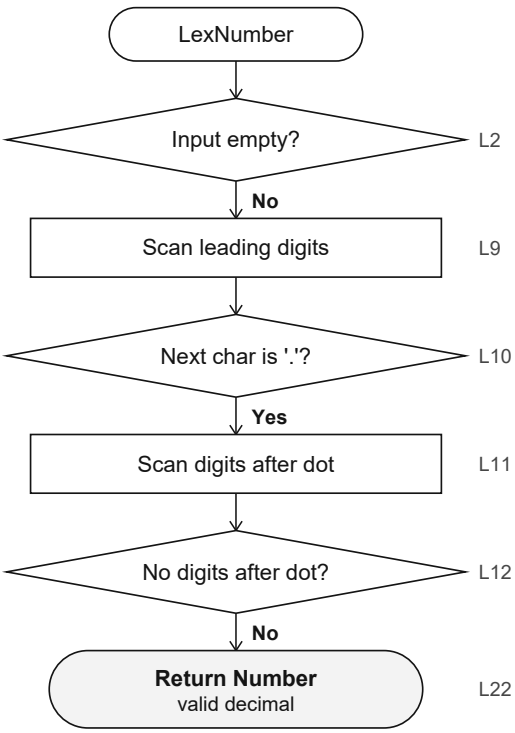


Fig. 4. Valid decimal. With `123.45x`, the token text and span cover `123.45`.

The other three returns have contracts too. `LexNumberEmptyIsInvalid` covers the empty input, `LexInvalidFloatEmptyMantissa` covers the dot with an empty fractional part, and `LexValidFloat` covers a dot followed by a nonempty digit run.

Table I. Results of running Listing 1 at line 0, column 0. Column pairs give the start and end of the token span.

Input	Kind	Token text	Columns
`""`	`Invalid`	`[FATAL] INVALID EMPTY NUMBER LEXEME`	`(0, 0)`
`"123x456"`	`Number`	`123`	`(0, 3)`
`"123.x"`	`Invalid`	`malformed floating point literal '123.'`	`(0, 4)`
`"123.45x"`	`Number`	`123.45`	`(0, 6)`

IV. PROVING THE INTEGER PATH

The integer theorem describes what the token represents. In Figure 2, the token must contain exactly the integer prefix, and its span must cover that prefix. Listing 2 states the conditions for this return and gives the whole token as the conclusion.

Listing 2. The statement of `LexValidInteger`.

```
theorem LexValidInteger
  (text intPart rest : List Char)
  (line column : Nat)
  (hintNonEmpty : 0 < intPart.length)
  (hallDigits : ∀ n ∈ intPart, Char.isDigit n)
  (hrestNoDot : rest[0]? != some '.')
  (hrestNoDigit : !rest[0]?.any Char.isDigit)
  (htext : text = intPart ++ rest) :
  LexNumber text line column =
    Token.Create TokenKind.Number
      (SourceLocation.Create
        (SourcePosition.Create line column)
        (SourcePosition.Create line
          (column + intPart.length)))
      (String.ofList intPart)
```

`htext` splits the input into `intPart ++ rest`. The prefix is nonempty and all its characters are digits. `hrestNoDigit` requires the remainder to be empty or to begin with a non-digit. `hrestNoDot` also rules out a dot at that boundary. After the first character of the remainder, anything can follow.

For `123x456`, the integer is `123` and the remainder is `x456`. The split `12` and `3x456` fails the first-character test. If we require the entire input to consist of digits, the theorem covers a standalone integer. By describing the remainder, we can also prove where the scan stops in a larger input.

Lean still considers the decimal branch when the proof splits on the code. The proof uses facts about `span` to identify the scanned prefix and the remainder. Listing 3 isolates one step. Dropping leading digits from `rest` leaves it intact. Once the scanner's remainder is identified with `rest`, `hrestNoDot` rules out the decimal branch.

Listing 3. A standalone Lean proof of the remainder fact used in the integer proof.

```
example (rest : List Char)
    (hrestNoDigit : !rest[0]?.any Char.isDigit) :
    List.dropWhile Char.isDigit rest = rest := by
  cases rest with
  | nil => rfl
  | cons ch tail => simp_all [List.dropWhile]
```

The contract gives the token and its span. The outer lexer uses the span to advance its cursor. The proof has to account for the input consumed and the state left for the next step.

V. DEFENSIVE BRANCHES AND CHANGES

A defensive branch may handle a state that is unreachable today. `ParseTopWithoutAtoms` specifies the diagnostic if a nonempty input ever produces no atoms. A separate theorem rules out that state in the current implementation. If a change makes it reachable, the conditional guarantee can still say how the caller should respond.

`OperatorClassificationMatchesIsOperator` illustrates another maintenance check. It states that the lexer's operator test agrees with an explicit character list. Lean can catch a change that makes the two disagree.

VI. COMPOSITION AND SCOPE

A local proof can give us a fact to use when reasoning about the caller. `Parse` uses `Lex`, and `Lex` uses `LexNumber` for a leading digit. `LexIsSound` uses `LexNumberIsSound` in that branch. The number theorem splits its own cases and shares scanner lemmas with `LexValidInteger`.

The tricky part is supplying the hypotheses. A leading digit is enough to establish the input conditions of `LexNumberIsSound`. To use the integer theorem directly, we would also have to identify the full digit prefix and show what follows it.

`LexIsSound` proves equality with the executable lexer specification. For an unrecognized character, the specification delegates the rest of the input to `Lex` itself. The parser proofs establish progress and specific return cases. A theorem relating the complete `Parse` function to a grammar would be a separate result.

VII. RELATED WORK

Symbolic execution accumulates a path condition from the branches taken with symbolic inputs [2]. In Averroes, Lean hypotheses state these conditions alongside facts about the input, such as its split into a digit prefix and remainder.

Lasser et al. [3] use Coq to verify an LL(1) parser generator and its outputs against an input grammar. Averroes illustrates path-level reasoning in a handwritten Lisp parser, using number lexing as the worked example.

REFERENCES

[1] Y. Daba, *Averroes*, Lean source code, rev. dcd46c5.

[2] J. C. King, "Symbolic execution and program testing," *Commun. ACM*, vol. 19, no. 7, pp. 385-394, 1976. doi: 10.1145/360248.360252.

[3] S. Lasser, C. Casinghino, K. Fisher, and C. Roux, "A verified LL(1) parser generator," in *Proc. ITP*, 2019, pp. 24:1-24:18. doi: 10.4230/LIPIcs.ITP.2019.24.